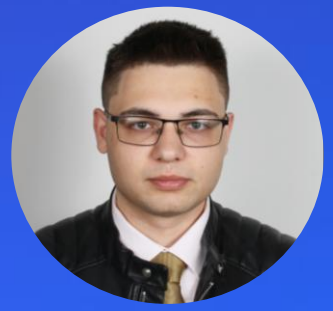


Alex Tsvetanov
Senior C++ Software Engineer

Async I/O with C++20 coroutines

One API over epoll, kqueue, IOCP and io_uring

Партньор: StorPool DISTRIBUTED STORAGE

Следете актуалните обяви за C, C++

DEV.BG



Where the time actually goes

1 / The problem



Threads do not scale

Thread per connection

Memory is the smaller cost.
The scheduler and the cache are the bigger one.

Past a few thousand connections you are paying for context switches you never asked for.

Thread pool + blocking calls

Moves the stall, does not remove it.

A blocked pool thread is a thread that cannot serve anyone else, and the queue behind it grows.

Most servers are not saturated.
The idle-to-medium path is where the overhead hides, and nobody profiles it.





Readiness versus completion

2 / Two families, four APIs

Two contracts, one job

Readiness / poll, epoll, kqueue

"This descriptor will not block."

The kernel tells you when. You then make the syscall yourself.

Nothing owns your buffer until you call.

Completion / IOCP, io_uring

"Here is the whole operation. Tell me when it is done."

The kernel runs it. io_uring adds a submission and completion ring in shared memory, so submissions batch.

The kernel owns your buffer while it runs.

Who owns the buffer while the operation is in flight?
That one question is why a naive wrapper over both is a use-after-free waiting for load.





C++20 coroutines

3 / Three functions and a frame

Партньор:


Следете актуалните обяви за C, C++

DEV.BG



The awaitable contract

await_ready

```
bool await_ready()
    const noexcept;
```

The fast path.
Return true when the answer is already known and skip the suspension entirely.

await_suspend

```
bool await_suspend(
  coroutine_handle<> h);
```

Where you hand control away.

Return false to carry on without suspending.

await_resume

```
int await_resume()
    const noexcept;
```

Produces the value the co_await expression evaluates to, once you are resumed.

When a coroutine suspends, its locals live in the frame, on the heap.
Not on a stack frame that is about to be reused. Same address, still alive,
until it resumes or is destroyed.



locals, packed for the trip



Putting them together

4 / The same four steps, two dialects



Read these side by side

Windows / IOCP

```
op.h = h;
WSABUF wb{len, buf};
DWORD flags = 0;
int r = WSARecv(
    s, &wb, 1, nullptr,
    &flags,
    &op.ov,   // find us here
    nullptr);
return true;   // suspended
```

Linux / io_uring

```
op.h = h;
sqe = io_uring_get_sqe(ring);
io_uring_prep_recv(
    sqe, s, buf, len, 0);
io_uring_sqe_set_data(
    sqe, &op);  // find us here
io_uring_submit(ring);
return true;   // suspended
```

Four steps, every time

1 / Remember

Store the coroutine_handle somewhere the completion can reach it.

op.h = h;

2 / Leave a trail

Point the kernel back at your handle, before it has the work.

OVERLAPPED on Windows, SQE user data on Linux. Same idea.

3 / Hand over

Give the kernel the whole operation: socket, buffer, length.

It owns the work, and your buffer, now.

4 / Suspend

Return true and stop running.

The completion event resumes you. Your handler never sees the event loop.

Where readiness differs

Linux / epoll

```
// await_suspend
op.h = h;
epoll_event ev{};
ev.events = EPOLLIN | EPOLLONESHOT;
ev.data.ptr = &op;   // find us here
epoll_ctl(ep, EPOLL_CTL_MOD,
          s, &ev);   // re-arm
return true;   // suspended

// await_resume: the I/O is here
return ::recv(s, buf, len, 0);
```

macOS, BSD / kqueue

```
// await_suspend
op.h = h;
struct kevent ev;
EV_SET(&ev, s, EVFILT_READ,
       EV_ADD | EV_ONESHOT,
       0, 0, &op);   // find us here
kevent(kq, &ev, 1, nullptr, 0, nullptr);
return true;   // suspended

// await_resume: the I/O is here
return ::recv(s, buf, len, 0);
```

How the wrapper breaks

One line, to save memory

```
// echo.cpp
char buf[kBufSize];

// naive.cpp
char* buf = pool.lend();

// the readiness assumption:
// free again once we return
```

Under load

One client: each receive finishes before anyone else wants the buffer. Every test passes.

A hundred idle clients: a hundred receives in flight, fifty per buffer. The kernel writes each arrival into memory another connection is about to send from.

Nothing crashes. Clients just get each other's bytes.



this is fine



char buf[kBufSize]; is an ordinary local.
It lives in the frame, so it is alive for exactly as long as the operation is. The buffer-lifetime problem does not get solved. It stops existing.

One condition: the frame must outlive the operation. That is why cancellation, next, is the hard part.

What still does not abstract

Cancellation

Four kernels, three stories.

CancelIoEx, IORING_OP_ASYNC_CANCEL, and on the readiness side simply not making the call.

There is no shared shape to hide behind.

Error surfaces

WSAGetLastError, errno, and a negative cqe->res.

Three vocabularies mapped onto one enum, by hand, and the mapping is where the bugs live.



Who resumes the coroutine?

5 / Scheduling, and what it costs

Two defensible answers

On the completion thread

Lowest latency. Best cache locality: the data is warm and you are already there.

One slow handler stalls every completion queued behind it.

Hand off to a pool

Isolation and parallelism.

Paid for with a queue hop and a cache miss per operation.

The queue between them is usually where the throughput goes. Instrument it first.



Demo

6 / One source, Windows and Linux

Live. What could possibly go wrong?





echo.cpp has no #ifdef in it at all.
Every platform difference lives in io.hpp. --trace shows the thread change while the buffer stays put.

naive.cpp: the same handler, one line changed. One client: it passes. A hundred: they get each other's bytes.



Pitfalls

7 / Four that cost people days



Four that cost people days

Dangling buffers

Someone helpfully hoists the buffer out of the frame into a caller.

Now it dies while the kernel is still writing to it.

Exceptions

A throw across the suspend boundary does not land where your instincts say.

Decide the policy before you need it.

The wrong thread

You get resumed on the completion thread.

Everything your handler touches has just become concurrent, whether you planned it or not.

Measuring at 100%

Full-load benchmarks hide the idle-path overhead completely.

Measure the load you actually run at.

Numbers you can trust

Tools

wrk and ab for throughput.

Percentiles, never averages. A mean latency hides exactly the tail you were hired to fix.

Always state the machine

Cores, NIC, kernel version, build flags.

A requests-per-second figure with no hardware attached is decoration, not evidence.



Coroute/WebFrame++, since 2017.
Unifies IOCP, io_uring and kqueue behind one coroutine API. It is also my doctorate, so I am being deliberately vague about the internals.

The published parts: TELECOM 2025, Optimizing Asynchronous Event Dispatch in Modern C++ Publish/Subscribe Systems. ICEST 2025, Modern C++ Publish/Subscribe Pattern.



Your turn.
Which backend do you run in production, and who owns your buffers while it waits? Unmute, or put it in the chat.



Thank you!

Contacts:

linkedin.com/in/alex-tsvetanov

github.com/Alex-Tsvetanov

